

CHATTY

Enterprise MERN Chat Application

Complete Technical Design & Architecture Document

Prepared by: Antigravity AI Partner

Date: 3/7/2026

Version: 2.0.0 (Enterprise Ready)

1. Executive Summary

Chatty is a secure, real-time, enterprise-ready messaging application built on top of the MERN (MongoDB, Express, React, Node.js) stack. The system provides instantaneous user-to-user communication with robust state synchronization, real-time typing indicators, read receipts, and user presence tracking. To meet enterprise standards, the application incorporates extensive security controls including request rate-limiting, secure Helmet headers, and NoSQL query sanitization.

2. Technology Stack

- Backend Server: Node.js with Express.js Framework
- Database: MongoDB (with Mongoose Object-Document Mapper)
- Real-Time Transport: Socket.io (WebSocket wrapper)
- Media Management: Cloudinary API Integration (for profile picture uploads)
- Frontend Client: React (Vite-powered SPA framework)
- State Management: Zustand (with hooks integration)
- Styling System: Tailwind CSS & DaisyUI Theme Framework

3. System Architecture & Real-Time Flow

The application follows a decoupled client-server architecture. The frontend application runs as a Single Page Application (SPA), sending HTTP REST API calls to the Node/Express backend server for state changes (e.g., authentication, settings, profile updates, and fetching chat history) and establishing a continuous bi-directional WebSocket connection via Socket.io for messaging actions.

Real-time events managed by the Socket.io server:

- **connection:** Triggered when a user opens the client. Binds the user's MongoDB ObjectId to their active socket instance and broadcasts the updated online user list to all subscribers.
- **disconnect:** Triggered when a user closes the tab or drops connection. Removes the socket mapping and updates the global online count.
- **typing:** Forwards typing status updates (isTyping: true/false) directly to the target recipient socket ID, prompting the "X is typing..." notification.
- **newMessage:** Emits the message object instantly to the receiver's socket ID. If the receiver has the chat window open, it triggers a read-receipt update.
- **messagesRead:** Signals the sender that the recipient has opened the conversation, updating ticks to blue.
- **messageDeleted:** Signals both client screens to filter out and remove a deleted message in real-time.

4. Database Schema Design

The data layer is managed in MongoDB with two primary collections: Users and Messages.

User Schema:

```
{
  email: { type: String, required: true, unique: true },
  fullName: { type: String, required: true },
  password: { type: String, required: true, minlength: 6 },
  profilePic: { type: String, default: "" },
  bio: { type: String, default: "Hey there! I am using ChatApp." }
} (plus automated timestamps)
```

Message Schema:

```
{
  senderId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  receiverId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  text: { type: String },
  image: { type: String },
  isRead: { type: Boolean, default: false }
} (plus automated timestamps)
```

5. Security & Reliability Controls

To meet enterprise readiness guidelines, the backend incorporates the following defensive mechanisms:

- **Helmet Security Headers:** Secures the server by setting various HTTP headers to guard against clickjacking, cross-site scripting (XSS), and other protocol-based exploits.
- **API Rate Limiting:** Limits each IP address to a maximum of 200 API requests per 15-minute window, effectively blocking brute-force authentication attacks and denial of service (DoS) attempts.
- **NoSQL query sanitization:** Custom middleware cleanses request parameters, queries, and bodies, automatically stripping keys starting with \$ or containing a dot (.) to block MongoDB operator injections.
- **Global Error Boundary:** Structured catch-all middleware intercepts runtime errors, logs the full execution stack trace via Winston, and serves a standardized HTTP 500 JSON response instead of crashing.
- **Structured Winston Logging:** Replaces standard logs with a structured JSON console output in production and colored streams in development, integrated with Morgan to track HTTP activity.

6. WhatsApp UI Transformation & UX Enhancements

The application user experience has been redesigned to align with the visual flow of WhatsApp Web:

- Sidebar Last Message Preview: Instead of a static list, contact rows show the last message content (or "📷 Photo"), the message timestamp, checkmark ticks, and unread counts.
- Unread Message Badge: A pulsing green circle showing unread count highlights conversations requiring attention, clearing instantly upon selecting the chat.
- Dynamic Sorting: The sidebar dynamically re-orders. Sending or receiving a message moves the active user to the top of the sidebar, mimicking active chat sorting.
- Custom Background Wallpaper: The message feed container implements WhatsApp's light beige (#f5f5dc) overlay in light mode and deep charcoal-blue (#0b141a) in dark mode.
- Speech Bubbles: Sent bubbles are styled in light green with sharp top-right corners, while received bubbles render in clean white/slate with sharp top-left corners.
- Floating Emoji Panel: An interactive emoji picker is embedded adjacent to the text box for rapid insertions.

